

MIPS Pipelined Processor with Hardware Performance Counters

-Anand S Menon

Modern processors rely on pipelining to execute multiple instructions simultaneously, improving speed by overlapping execution stages. However, this introduces hazards — data dependencies, control flow changes, and structural conflicts, that can stall or corrupt execution. Understanding how and when these hazards occur is critical to evaluating real processor performance, and that's exactly what hardware performance counters are designed to do.

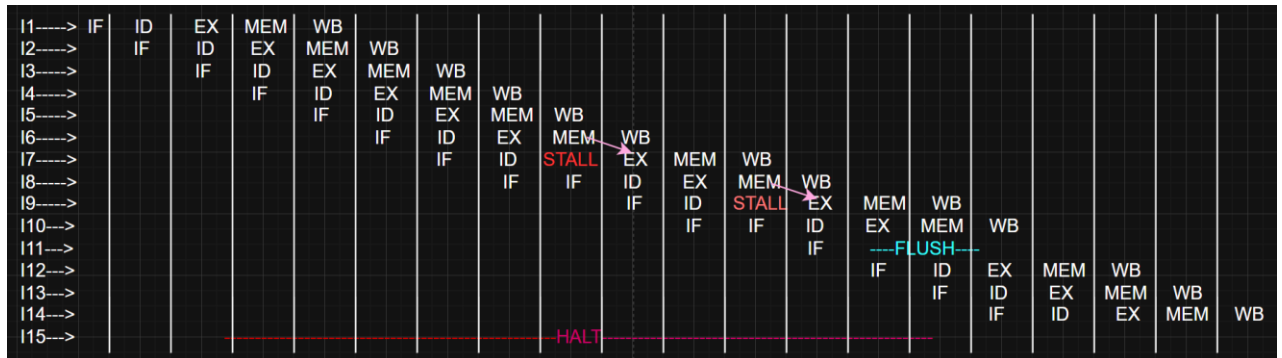
This project implements a Hardware Performance Counters on a 5-stage pipelined MIPS processor in SystemVerilog, tracking key performance events such as instruction retirement, memory accesses, branch mispredictions, and pipeline stalls in real time.

To better understand how these counters behave in practice, consider the following example program :

HEXADECIMAL :

2010000a	I1:addi \$s0, \$zero, 10 # \$s0 = 10
2011000a	I2:addi \$s1, \$zero, 10 # \$s1 = 10
20120005	I3:addi \$s2, \$zero, 5 # \$s2 = 5
ac100000	I4:sw \$s0, 0(\$zero) # mem[0] = 10 (mem access 1)
ac110004	I5:sw \$s1, 4(\$zero) # mem[1] = 10 (mem access 2)
8c080000	I6:lw \$t0, 0(\$zero) # \$t0 = 10 (mem access 3, load-use stall)
01124820	I7:add \$t1, \$t0, \$s2 # \$t1 = 15 (stall here)
8c0a0004	I8:lw \$t2, 4(\$zero) # \$t2 = 10 (mem access 4, load-use stall)
01525822	I9:sub \$t3, \$t2, \$s2 # \$t3 = 5 (stall here)
12110001	I10:beq \$s0, \$s1, +1 # TAKEN (10==10) → mispredict, skip next
20130063	I11:addi \$s3, \$zero, 99 # SKIPPED
2013002a	I12:addi \$s3, \$zero, 42 # \$s3 = 42
11320001	I13:beq \$t1, \$s2, +1 # NOT TAKEN (15!=5) → correct prediction

```
fc000000    HALT
```



REPORT:

```
=====
                        IE2025005 -- PMU Performance Report
=====
Pipeline: 5-stage in-order, static not-taken
-----
EXECUTION SUMMARY
CPU Cycles           : 21
Instructions Retired : 13
CPI                  : 1.62
-----
MEMORY
Memory Accesses      : 4
Memory Access Rate   : 30.8%
-----
BRANCH ANALYSIS
Branch Count         : 2
Branch Frequency     : 15.4%
Mispredictions       : 1
Prediction Accuracy  : 50.0%
-----
HAZARD ANALYSIS
Load-Use Stall Cycles: 2
Branch Flush Cycles  : 1
Useful Cycles        : 18
Pipeline Efficiency   : 85.7%
=====
```

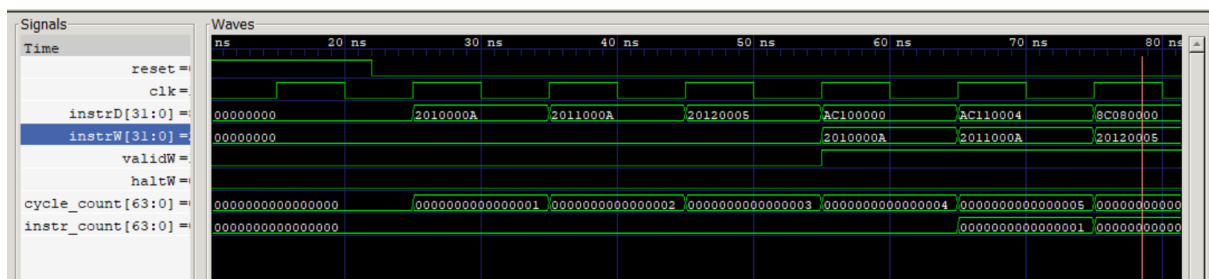
Now let's examine each counter individually, tracing the pipeline signals that drive them and understanding how each value is derived during execution.

EXECUTION COUNTERS (Instructions retired, Cycle count)

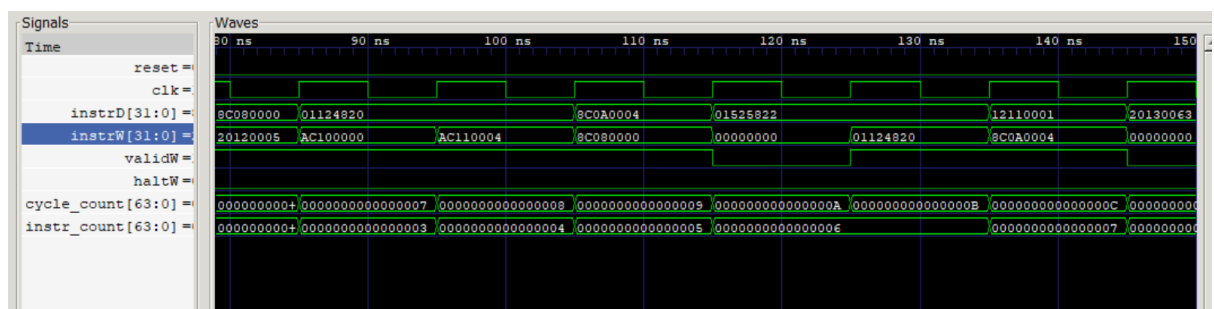
We bring in 2 extra signals alongside every instruction through the pipeline - valid and halt.

Valid is a flag that tells us whether what's currently in Writeback is a real instruction or just a bubble from a stall or flush.

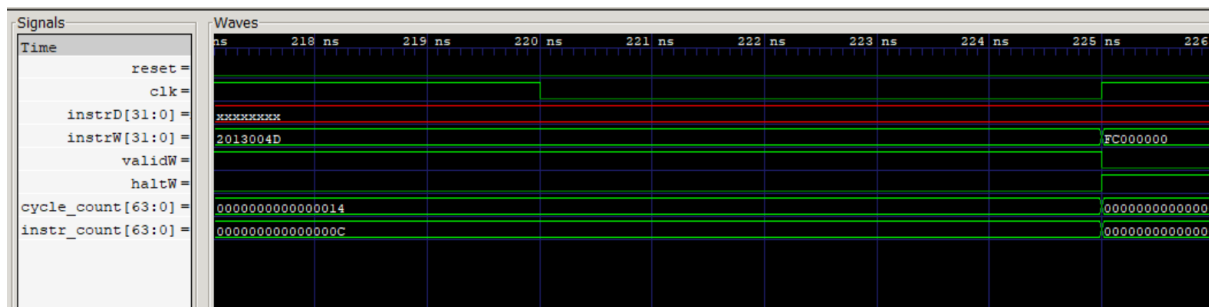
halt goes high when the HALT instruction reaches Writeback, which is when the testbench knows the program is done and prints the results.



- instrD and instrW both show 00000000 (bubbles) and both cycle_count and instr_count sit at 0. Once reset goes low at around 25ns, the already loaded (IF stage) instructions begin flowing in. I've added the instrW signal to show the rising of the validW signal.
- We can observe validW rising after the instruction passes into WB and the instruction_count incrementing in the next cycle.



- Observe 01124820 – instruction 7, 1st stall
- When a stall happens, flushE = 1 which clears validE to 0. A bubble is inserted into Execute. That bubble propagates through EX→MEM→WB with valid = 0 the whole way. So when it reaches WB stage, validW = 0 and instr_count doesn't increment.
- Meanwhile the real instruction (i7) is held in Decode with stallD = 1, so its validD stays 1 — it just waits one extra cycle before moving forward.



When FC000000 – HALT enters WB stage, haltW goes high and immediately stops the counters.

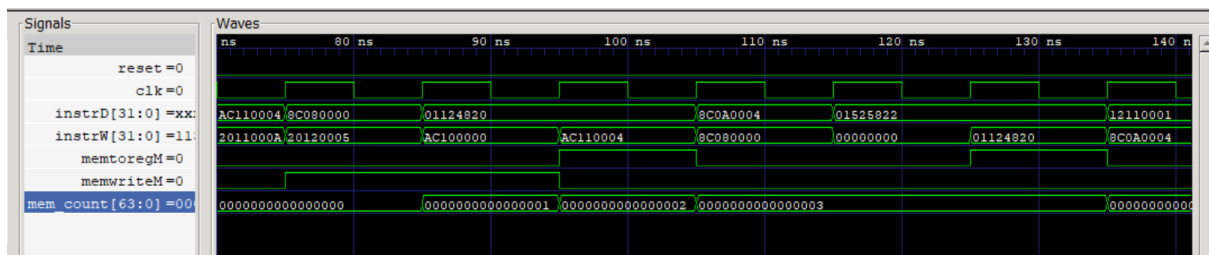
MEMORY COUNTERS (Memory access count)

To detect Memory Access, we use the memtoReg and memwrite signals.

memtoRegM goes high when a lw instruction is in the Memory stage. It tells the pipeline to route the data memory output back to the register file.

memwriteM goes high when a sw instruction is in the Memory stage, it enables the write enable on data memory.

The PMU ORs these two signals together, so mem_count increments whenever either a load or a store is passing through the Memory stage, giving a total count of all data memory accesses.



mem_count only increments at the next posedge. So there's always a 1 cycle gap between the signal going high and the counter updating.

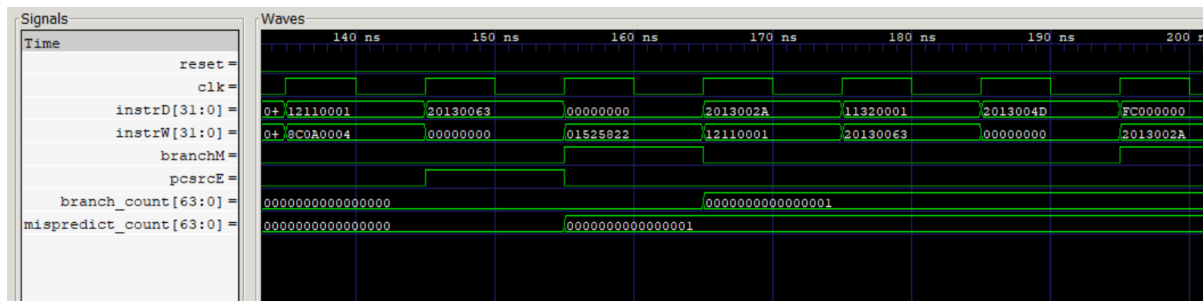
BRANCH COUNTERS (branch counts, misprediction count for static not taken)

Here, we utilise the Branch and pcsrc signals.

branchM goes high when a branch instruction reaches the Memory stage. This is when branch_count increments — it counts every branch regardless of whether it was taken or not.

pcsrcE goes high in Execute when a branch is both a branch instruction AND the ALU result is zero, meaning the branch is TAKEN. Since this pipeline uses a static not-taken

predictor, every taken branch is a misprediction. So mispredict_count increments whenever pcsrcE is high, and pcsrcE is also the signal that triggers flushD to clean up the wrongly fetched instruction.



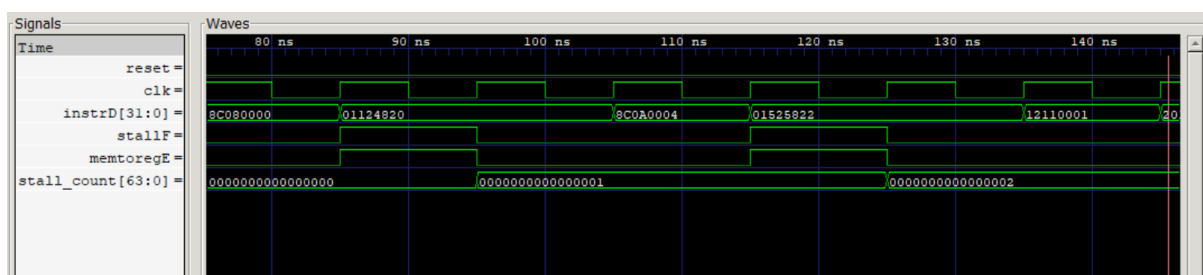
- Pcsr=1 indicates branch taken. Therefore, misprediction count increments in the next positive edge.

STALL COUNTERS (number of stalls)

We route a signal called stall through all stages.

memtoRegE goes high when a lw instruction is in Execute. The hazard unit checks if the destination register of that lw matches either source register of the instruction in Decode.

If it does, stallF and stallD go high for one cycle. stall_count simply increments every cycle that stallF is high, giving a direct count of how many cycles were wasted waiting for a load result.



memtoRegE = 1 means there's a load in Execute, and stallF = 1 means the hazard unit detected a dependency and is stalling the pipeline.

Beyond the raw counter values, several derived metrics can be computed to get a fuller picture of pipeline performance.

- CPI (Cycles Per Instruction) = $\text{cycle_count} / \text{instr_count}$. The closer this is to 1.0, the more efficiently the pipeline is running.

- Pipeline efficiency is derived as $(\text{cycles} - \text{stalls} - \text{mispredictions}) / \text{cycles}$, giving the percentage of cycles that were actually doing useful work.
- Branch prediction accuracy is calculated as $(\text{branch_count} - \text{mispredict_count}) / \text{branch_count}$, showing how often the static not-taken predictor guessed correctly